ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

Lecture 5: Semi-Decidable Problems, Complexity, P, NP, Reducibility

Halting Problem

In computability theory, the halting problem is the problem of determining, from a description of an arbitrary computer program and an input, whether the program will finish running, or continue to run forever.

$$\mathrm{H} = \begin{cases} w\&x \mid < T > = w \text{ and} \\ T \text{ stops when } x \text{ is entered as input} \end{cases}$$

Is χ_H *computable*?

- ⇨ Alan Turing proved in 1936 that a general algorithm to solve the halting problem for all possible program-input pairs cannot exist.
- ⇨ The halting problem is semi-decidable.

Post Correspondence Problem

$$PCP: x = \langle (v_1, w_1), (v_2, w_2), \dots, (v_k, w_k) \rangle$$

$$v_1, w_1 \in \sum^{+},$$

$$\exists \mathrm{i}_1, \mathrm{i}_2, \dots, \mathrm{i}_\mathrm{l} \in \{1, \dots, k\} \text{ with}$$

$v_{i_1} v_{i_2} \dots v_{i_l} = w_{i_1} w_{i_2} \dots w_{i_l}$? (if the equality is true, then we have a **solution**!)

$$\mathrm{PCP}_{\sum} = \{x_k \mid x_k \text{ has solution } \mathrm{i}_1, \mathrm{i}_2, \dots, \mathrm{i}_\mathrm{l} \in \{1, \dots, k\}\}$$

Is χ_{PCP} *computable*?

- ⇨ The PCP is another undecidable decision problem introduced by Emil Post in 1946
- ⇨ The PCP is semi-decidable.

Consider the following two lists:

α_1	α_2	α_3
a	*ab*	*bba*

β_1	β_2	β_3
baa	*aa*	*bb*

A solution to this problem would be the sequence (3, 2, 3, 1), because

$$\alpha_3\alpha_2\alpha_3\alpha_1 = \mathrm{bba} \circ \mathrm{ab} \circ \mathrm{bba} \circ \mathrm{a} = \mathrm{bbaabbbaa} = \mathrm{bb} \circ \mathrm{aa} \circ \mathrm{bb} \circ \mathrm{baa} = \beta_3\beta_2\beta_3\beta_1$$

Furthermore, since (3, 2, 3, 1) is a solution, so are all of its "repetitions", such as (3, 2, 3, 1, 3, 2, 3, 1), etc.; that is, when a solution exists, there are infinitely many solutions of this repetitive kind.

However, if the two lists had consisted of only α_2, α_3 and β_2, β_3 from those sets, then there would have been no solution (the last letter of any such α string is not the same as the letter before it, whereas β only constructs pairs of the same letter).

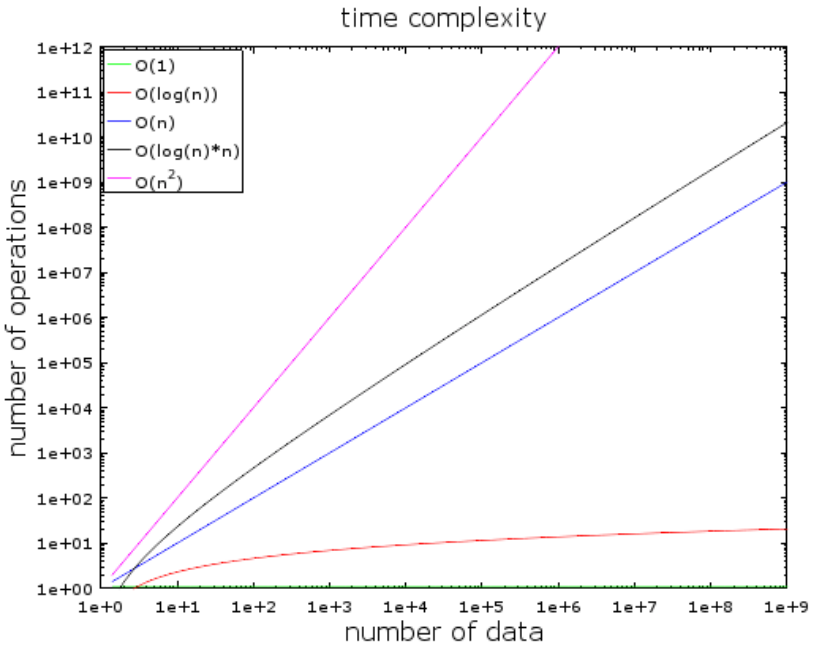
⇨ Both the halting problem and PCP are often used in proofs of undecidability.

Complexity

- Time complexity
- Space complexity
- Communication complexity

O-Notation: $f \in O(g)$

$$O(1), O(logn), O(n), O(nlogn), O(n^k)$$



1E+9	1,000,000,000 (giga)
1E+6	1,000,000 (mega)
1E+3	1,000 (kilo)
1E+2	100 (hector)
1E+1	10 (deka)
1E+0	1 (-)

Deterministic TM: $time_T: \sum^* \rightarrow \mathbb{N}$

$time_T(w) =$ Number of configuration transitions when processing w

$$f: \mathbb{N}_0 \rightarrow \mathbb{N}_0$$

$$TIME(f) = \{L \subseteq \sum^* \mid \exists det.\ TM\ T\ with\ L = L(T)\ with\ time_T(w) \in O(f(|w|))\}$$

Example: $TIME(n^2)$

$$P = \bigcup_{k \geq 0} TIME(n^k)$$

↓

Class of languages that are decided by *deterministic* Turing machines in polynomial time

Non-Deterministic TM: $ntime_T: \sum^* \rightarrow \mathbb{N}$

$ntime_T(w) = min(\text{Number of configuration transitions when processing } w)$

$$f: \mathbb{N}_0 \rightarrow \mathbb{N}_0$$

$$NTIME(f) = \{L \subseteq \sum^* \mid \exists non-det.\ TM\ T\ with\ L = L(T)\ with\ ntime_T(w) \in O(f(|w|))\}$$

Example: $NTIME(n^2)$

$$NP = \bigcup_{k \geq 0} NTIME(n^k)$$

↓

Class of languages that are decided by *non-deterministic* Turing machines in polynomial time

Question: $P =^? NP$

It is assumed that P is not equal to NP.

$$DTM_{\sum} = NTM_{\sum}$$

$$EXPTIME = \bigcup_{c>1} TIME(c^n)$$

$$P \subseteq NP \subseteq EXPTIME \subseteq NEXPTIME$$

Reducibility

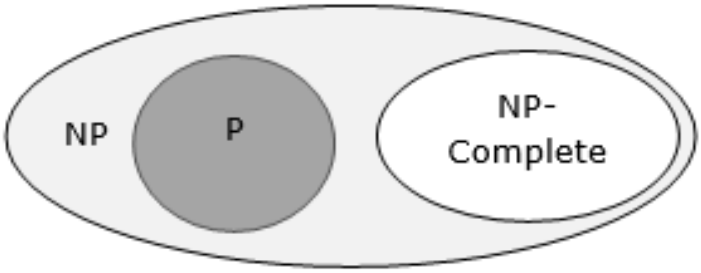
$L_1 \subseteq \sum_1^*$ polynomially reducible to $L_2 \subseteq \sum_2^*$ iff there is a total function $f: \sum_1^* \rightarrow \sum_2^*$ that is computable in polynomial time: $w \in L_1$ iff $f(w) \in L_2$

$L_1 \leq L_2$ *or* $L_1 \leq_{pol} L_2$

- transitive, $L_1 \leq L_2,\ L_2 \leq L_3\ \Rightarrow\ L_1 \leq L_3$
- $L_2 \in P, L_1 \leq L_2 \Rightarrow\ L_1 \in P$
- $L_2 \in NP, L_1 \leq L_2 \Rightarrow\ L_1 \in NP$
- $L_2 \notin P, L_1 \leq L_2 \Rightarrow\ L_1 \notin P$
- $L_2 \notin NP, L_1 \leq L_2 \Rightarrow\ L_1 \notin NP$

NP-easy, NP-hard, NP-complete

- $L \subseteq \sum^*$ is called NP-easy iff $L \in NP$
- $L \subseteq \sum^*$ is called NP-hard iff $L^\iota \leq L,\ \forall L^\iota \in NP$
- $L \subseteq \sum^*$ is called NP-complete (NPC) iff L is NP-easy and NP-hard.



Note: L ∈ NPC. Then P = NP iff L ∈ P

Methods to show that L is in NPC:

(1) (i) Show $L \in$ NP
(ii) Show $L^\iota \leq L,\ \forall L^\iota \in NP$

(2) (i) Show $L \in$ NP
(ii) Show $L^\iota \leq L,\ for\ a\ single\ L^\iota \in NPC$

$L^\iota \in NPC \rightarrow L^{\iota\iota} \leq L^\iota \leq L,\ \ L^{\iota\iota} \in NP$

Examples of Reduction:

$$SAT \in NPC\ (S.Cook)$$

$$SAT \leq 3SAT \leq HAMILTON \leq TSP$$

$$3SAT \leq CLIQUE$$

$$3SAT \leq MAX2SAT \leq NAESAT \leq 3CG$$

$$3SAT \leq KNAPSACK \leq PARTITION \leq BINPACKING$$